Eval-Driven Document Q&A System

Phase-wise Workflow and System Design Foundation

A practical project blueprint for AI Engineer, Data Engineer, and Data Scientist roles

Project type	Eval-driven document intelligence / grounded Q&A system
Primary goal	Ingest unstructured documents, retrieve evidence, answer with citations, and measure quality through evaluations

This document explains the project idea, the complete phase-wise development plan, and the base system design to use as a build reference.

1. Project Overview

The project is an AI-powered document question-answering system that works on a controlled set of uploaded documents such as tenders, policies, contracts, manuals, or annual reports. Instead of behaving like a generic chatbot, it retrieves relevant evidence from the uploaded files, generates grounded answers, attaches citations, logs system behavior, and evaluates answer quality systematically.

The business value is not that it uses an LLM. The value is that it wraps the LLM inside an engineered system with ingestion, retrieval, evidence control, observability, and evaluation. This makes the project suitable for interviews across AI Engineer, Data Engineer, and Data Scientist roles.

Why this project is strong for a resume

- It demonstrates end-to-end system thinking rather than a single notebook or API call.
- It combines data ingestion, storage design, retrieval, generation, and evaluation in one coherent project.
- It can be built to a high quality without taking months if the scope is kept narrow.
- It is easy to explain in interviews because the business problem is clear: find and answer from large documents reliably.

Core objective

Build a system that ingests documents, converts them into searchable knowledge, retrieves the best evidence for a user query, generates grounded answers with page-level citations, stores logs for analysis, and uses an evaluation set to measure and improve retrieval and answer quality.

2. System Design Foundation

The project should be treated as a layered application. The model is only one layer. The real system value comes from the surrounding architecture.

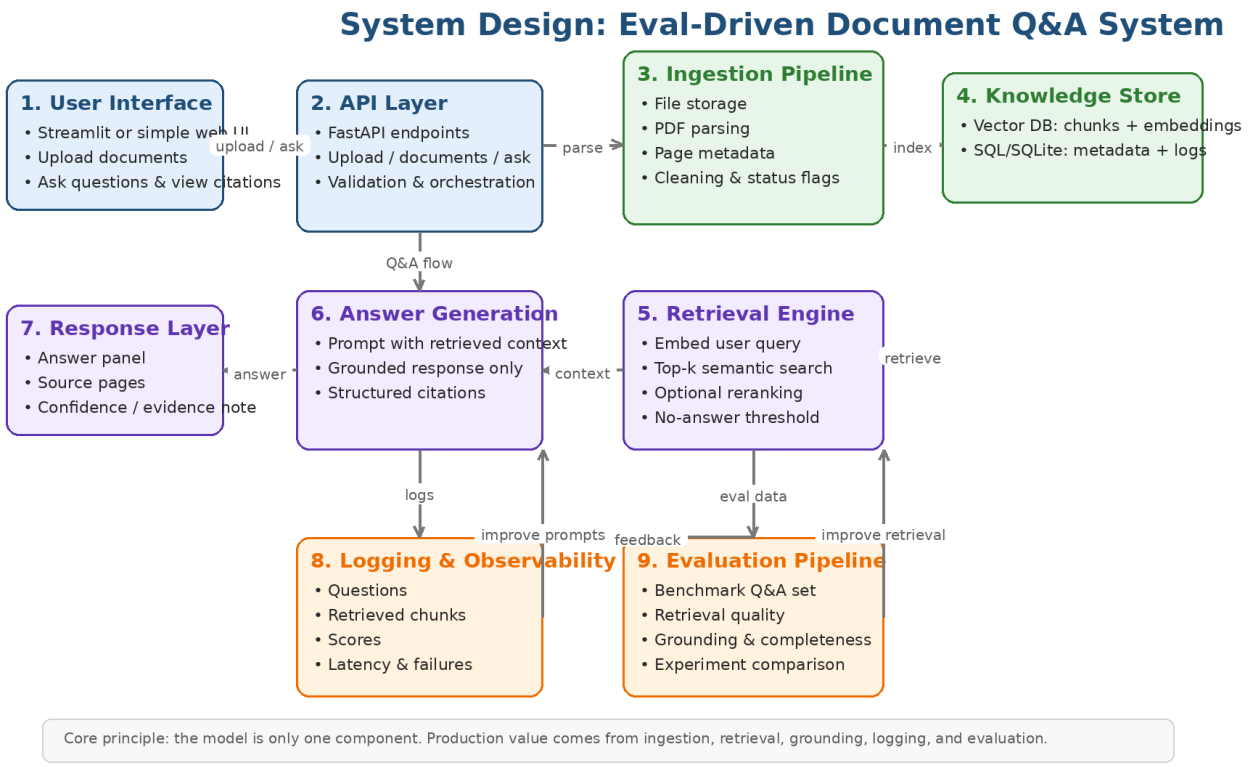

Figure 1. High-level system design for the Eval-Driven Document Q&A System.

System layers

User / UI layer: Accepts document uploads, displays document inventory, collects user queries, and renders answers with citations.

API layer: Acts as the controlled gateway for uploads, indexing requests, search requests, and answer generation requests.

Ingestion layer: Stores files, parses PDFs page by page, cleans text lightly, and attaches metadata such as page number, file name, and parse status.

Knowledge layer: Chunks text into semantically meaningful units, generates embeddings, stores vectors, and maintains metadata references.

Retrieval and reasoning layer: Embeds the query, retrieves top-k candidate chunks, optionally reranks them, and prompts the LLM to answer only from retrieved evidence.

Observability and evaluation layer: Stores logs, benchmark questions, retrieval results, and experiment outcomes so quality can be measured and improved.

Core data flow

- User uploads a PDF or document.
- The file is stored and registered in the metadata database.
- The parser extracts page-level text and stores parse output.
- The preprocessing stage cleans text and creates chunks.
- Embeddings are generated and stored in a vector index.

- The user asks a question.
- The retrieval engine finds the most relevant chunks.
- The LLM answers using only the retrieved context.
- The answer is returned with citations and the full interaction is logged.
- The evaluation pipeline compares expected and actual system behavior and guides improvements.

3. Recommended Scope and Assumptions

To keep the build precise and time-efficient, the scope should be narrowed to one document domain. A narrow domain increases answer quality, simplifies evaluation, and makes the final story much stronger.

Recommended domain choices

Domain	Why it works	Interview strength
Tender / specification documents	Information is dense and question-oriented, which is ideal for retrieval and citations.	Strong for AI + data roles because the use case is practical and high-value.
Policy / SOP documents	Easy to benchmark because questions often map to explicit clauses or procedures.	Shows enterprise search and knowledge management value.
Technical manuals	Great for section-level retrieval and troubleshooting queries.	Useful for grounded Q&A and error reduction.
Annual reports	Good for long-document parsing and section-aware retrieval.	Strong for document intelligence and metadata extraction.

MVP boundary

The first strong version should not include enterprise authentication, multi-tenant permissions, fancy dashboards, or complex OCR pipelines across all files. The MVP should focus on one user flow: upload documents, index them, ask questions, get grounded answers with citations, and evaluate quality.

4. Phase-wise Workflow

The workflow below is the recommended build order. Each phase is explained in terms of objectives, key tasks, outputs, and why it matters.

Phase 0 — Problem framing and scope lock

This phase prevents scope creep. Without a fixed domain, the project becomes generic and difficult to evaluate. It also helps determine the correct metadata fields, question patterns, and evaluation design.

Key work items

- Choose one document domain and freeze the problem statement.
- Define target user, document types, and the top question categories the system must answer.
- Write a success definition for the MVP such as: answer must include citations and avoid unsupported claims.
- Prepare 5–10 sample documents that will be used throughout development and testing.

Expected output

Deliverable: A frozen problem statement, selected document domain, sample document set, and a definition of success for the MVP.

Phase 1 — Data ingestion and document parsing

This phase creates the raw-to-structured foundation. If parsing is poor, every later phase suffers. The output of this phase should be a reliable page-level representation of each document plus diagnostics.

Key work items

- Build the upload flow and store raw files safely.
- Register each document with metadata such as document id, filename, file path, upload time, parser used, page count, and status.
- Parse documents page by page using a primary parser such as PyMuPDF.
- Store raw page text, lightly cleaned text, page number, character count, and parse warnings.
- Flag weak pages that may need OCR or manual review.

Expected output

Deliverable: Raw file storage, document metadata records, page-level parsed output, and parse diagnostics.

Phase 2 — Text cleaning, chunking, and metadata modeling

This phase transforms large page blobs into searchable units. Chunking quality directly affects retrieval quality, so the design should preserve meaning and traceability.

Key work items

- Normalize spacing and blank lines without destroying useful structure.
- Detect sections or headings where possible.
- Split text into semantically coherent chunks with page references and chunk order.
- Design the chunk metadata schema, including document id, page number, section title, chunk id, and text.

Expected output

Deliverable: Chunked document representation with a stable metadata schema.

Phase 3 — Embeddings and knowledge indexing

This phase creates the searchable knowledge layer. The key design requirement is that every vector record can be traced back to a document, page, and chunk.

Key work items

- Generate embeddings for every chunk.
- Store embeddings in a vector database or local vector index.
- Maintain a mapping between vector records and the metadata store.
- Add re-indexing support so updated documents can be reprocessed.

Expected output

Deliverable: A vector index populated with embeddings and linked back to metadata.

Phase 4 — Retrieval pipeline

This phase is the search engine of the project. A strong retrieval layer often matters more than prompt wording because answer quality depends on the evidence sent to the model.

Key work items

- Embed the user query.
- Retrieve the top-k most relevant chunks using semantic search.
- Optionally rerank the retrieved chunks before answer generation.
- Apply a no-answer threshold if the evidence is weak.

Expected output

Deliverable: A query-to-context retrieval function that returns relevant chunks and confidence signals.

Phase 5 — Grounded answer generation

This phase turns evidence into a user-facing response. The objective is not creativity. The objective is grounded, auditable answers.

Key work items

- Build a prompt template that instructs the model to answer only from retrieved context.
- Generate a concise answer plus page-level citations.
- Return a fallback response when evidence is insufficient.
- Standardize output formatting so every answer is easy to read and inspect.

Expected output

Deliverable: A grounded answer function that returns concise responses and citations.

Phase 6 — API layer and user application

This phase makes the system usable. The UI can remain simple. The real value is in the backend behavior and evidence display.

Key work items

- Create API endpoints such as upload, list documents, ask question, and retrieve logs.
- Build a minimal UI, preferably Streamlit or a lightweight web interface.
- Display the answer, citations, and source snippets clearly.
- Allow users to inspect which documents are available in the corpus.

Expected output

Deliverable: A usable application flow with upload, ask, and inspect capabilities.

Phase 7 — Logging and observability

This phase is what separates a demo from a real system. Logging allows debugging, performance tuning, and interview-ready discussions on reliability.

Key work items

- Log every question, retrieval result, scores, answer, latency, and failure mode.
- Store chunk ids retrieved for each query.
- Track when the system returns no-answer or low-confidence outcomes.
- Create basic analysis views such as top failure categories or slowest queries.

Expected output

Deliverable: Structured logs and basic operational visibility.

Phase 8 — Evaluation framework and experiments

This is the strongest phase for recruiter credibility. It proves the project was measured and improved rather than merely assembled.

Key work items

- Create a benchmark set of domain-specific questions and expected source references.
- Score retrieval quality: did the system retrieve the right chunk or page?
- Score answer quality: correctness, grounding, completeness, and brevity.
- Run small experiments by changing chunk size, top-k, prompt wording, or reranking strategy.
- Document before-and-after results.

Expected output

Deliverable: An evaluation dataset, metrics, and documented experiments.

Phase 9 — Packaging, documentation, and portfolio presentation

This phase turns a codebase into a portfolio asset. The project should be easy to understand without opening every file in the repository.

Key work items

- Prepare a clear README with setup steps, architecture, and screenshots.
- Add diagrams, sample queries, and a summary of evaluation results.

- Create 3–5 concise resume bullets and a short project explanation for interviews.
- Record known limitations and future improvements.

Expected output

Deliverable: A polished GitHub-ready portfolio project with supporting documentation.

5. Detailed Phase Notes and Implementation Guidance

Phase 1 implementation notes

- Store each uploaded file under a unique internal name rather than the original filename alone.
- Save both raw page text and lightly cleaned text so debugging remains easy.
- Do not over-clean. Preserve page boundaries, headings, and natural structure.
- Mark pages with extremely low text length for OCR consideration instead of silently treating them as valid.
- Use one document table and one page table at minimum.

Completion signal: This phase should end only when 3–5 different document types can be parsed successfully without silent failures.

Phase 2 implementation notes

- Chunk by meaning, not only by fixed length. A heading and its explanation should usually stay together.
- Include page number and chunk order in every record.
- Avoid chunks that are too large for retrieval and too small to contain useful context.
- Preserve section titles where possible because they become powerful retrieval metadata.

Completion signal: This phase should end with chunk records that are human-inspectable and clearly traceable back to source pages.

Phase 3 implementation notes

- Choose a simple vector store first, such as FAISS or Chroma, if the goal is speed.
- Keep the embedding pipeline deterministic so re-indexing the same document produces stable metadata mapping.
- Store index build timestamps and document versions for debugging.

Completion signal: This phase should end with successful indexing and the ability to inspect vector-linked metadata.

Phase 4 and 5 implementation notes

- Treat retrieval and generation as separate modules so each can be tuned independently.
- Never let the generation module answer without evidence.
- Return source page references in a standardized format.

• Add a no-answer behavior early. This makes the system more trustworthy than always producing a fluent answer.

Completion signal: These phases should end with reliable query handling on sample documents and answers that can be verified manually.

Phase 7 and 8 implementation notes

• Store enough detail to replay a query session later.

• Use a small but meaningful benchmark set; 30 to 50 questions is enough for an MVP.

• Measure both retrieval quality and answer quality, because a wrong answer may come from bad retrieval or bad reasoning.

Completion signal: These phases should end with a quality report that shows which changes improved the system.

6. Suggested Technical Stack

The technical stack should be practical, lightweight, and fast to build. Over-engineering reduces the chance of finishing a polished project.

Layer	Recommended choice	Reason
Backend API	FastAPI	Simple, production-friendly, and easy to explain.
Frontend	Streamlit or lightweight web UI	Fastest way to create a usable interface.
PDF parsing	PyMuPDF	Strong page-level extraction and practical for long PDFs.
Vector store	FAISS or Chroma	Good balance between ease and functionality for an MVP.
Metadata/logging DB	SQLite initially, PostgreSQL later	SQLite is fast to start; PostgreSQL is stronger for production-style design.
LLM / embeddings	OpenAI models or equivalent	Supports embeddings, answer generation, and grounded prompting.
Experiment tracking	CSV logs, SQLite, or MLflow-lite approach	Enough to compare configs without adding too much complexity.

7. Minimal Data Model

The project should define a small but clean schema from the beginning. This prevents later refactoring when logs and citations become important.

documents: doc_id, original_filename, stored_path, upload_time, page_count, parse_status, parser_used, domain_tag

document_pages: page_id, doc_id, page_no, raw_text, cleaned_text, char_count, parse_warning, needs_ocr

chunks: chunk_id, doc_id, page_no, section_title, chunk_order, chunk_text, embedding_id

query_logs: query_id, question, retrieved_chunk_ids, retrieval_scores, answer_text, latency_ms, created_at

eval_questions: eval_id, question, expected_doc, expected_page, expected_answer, eval_type

8. Recommended Build Order

1. Freeze the domain and success criteria.

2. Implement upload and page-level parsing.

3. Design and verify chunking output.

4. Create the embedding and indexing pipeline.

5. Implement retrieval and inspect top-k quality manually.

6. Implement grounded answer generation with citations.

7. Add logging for every query.

8. Create a benchmark question set and evaluate baseline performance.

9. Tune chunking, top-k, and prompts based on eval results.

10. Package the project with README, diagrams, and resume bullets.

9. Risks, Constraints, and Practical Guardrails

• Do not present the project as a generic PDF chatbot. Present it as an eval-driven document intelligence system.

• Do not skip citations. Without citations, the trust story becomes weak.

• Do not skip logging. Without logs, the project looks like a demo rather than a system.

• Do not skip evaluation. Evals are the main defense when a recruiter asks what makes the project more than a wrapper around an LLM.

• Do not overbuild the frontend. Put effort into parsing, retrieval, grounding, and quality measurement instead.

10. Final Reference Summary

This project is best understood as a production-style document intelligence pipeline. It takes uploaded documents, converts them into structured and searchable knowledge, retrieves relevant evidence for a question, generates grounded answers with citations, logs system behavior, and improves quality through evaluation and experiments.

For a resume, the strongest project story is not that the model answered questions. The strongest story is that the system was engineered to be scoped, reliable, measurable, and useful for a real workflow.

Use this document as the build foundation. Once the project starts, each phase can be converted into implementation tickets, folder structure, and code modules.